

Katholieke
Universiteit
Leuven

**Department of
Computer Science**

GEAVANCEERDE COMPUTERARCHITECTUUR

Labo 2

Tano Pannekoucke (r0955339)

Academic year 2025-2026

Inhoudsopgave

1 Inleiding **2**

2 Implementatie Strategieën **2**
2.1 GPU Atomic Approach 2
2.2 GPU Reduction Approach 2
2.3 Grid-Stride Loop (Schaalbaarheid) 2

3 Resultaten en Analyse **2**
3.1 Scaling Analysis (Time vs. Input Size) 2
3.2 Atomic vs. Reduction Analyse 3
3.3 Stabiliteitsanalyse 3

4 Antwoorden op Guiding Questions **4**
4.1 Is the result correct? 4
4.2 How does time increase as a function of input size? 4
4.3 What happens if the input is larger than 2048? 4
4.4 Consider calling the kernel with much less threads than data elements. 4
4.5 How would you tackle a 2D array? 4

5 Conclusie **5**

A Bijlage: Broncode **6**

1 Inleiding

Het doel van dit labo was het bestuderen van **race conditions** en het implementeren van parallelle algoritmen op de GPU om deze te vermijden. Een race condition ontstaat wanneer meerdere threads gelijktijdig schrijven naar, of lezen van, hetzelfde geheugenadres, wat leidt tot onvoorspelbare resultaten.

We hebben drie methoden geïmplementeerd en vergeleken voor het vinden van het maximum in een array ("Max Detection"):

1. **CPU Baseline:** Een sequentiële implementatie met een lus.
2. **GPU Atomic Operations:** Gebruik van `atomicMax` om toegang tot het resultaat te beveiligen.
3. **GPU Reduction:** Een parallel boom-structuur algoritme met shared memory.

2 Implementatie Strategieën

2.1 GPU Atomic Approach

Deze methode maakt gebruik van de ingebouwde CUDA-functie `atomicMax`. Een atomaire operatie zorgt ervoor dat de instructie (lezen, vergelijken, schrijven) ononderbroken wordt uitgevoerd. Hierdoor krijgen threads virtueel sequentieel toegang tot de gemeenschappelijke variabele, wat race conditions effectief voorkomt.

2.2 GPU Reduction Approach

Parallelle reductie is een techniek waarbij de dataset iteratief wordt verkleind. De complexiteit van dit algoritme is $\mathcal{O}(\log_2 N)$.

- **Shared Memory:** Om de bandbreedte naar het globale geheugen te minimaliseren, wordt data eerst ingeladen in shared memory (`extern __shared__ int sdata[]`).
- **Synchronisatie:** Tussen de reductiestappen in is synchronisatie vereist. We gebruiken `__syncthreads()` om te garanderen dat alle threads in een block hun berekening hebben voltooid voordat we verdergaan.

2.3 Grid-Stride Loop (Schaalbaarheid)

Een belangrijk onderdeel van de implementatie is de afhandeling van inputs die groter zijn dan het aantal beschikbare threads ($N >$ Threads). Hiervoor is een **Grid-Stride Loop** geïmplementeerd:

```
1 for (int i = idx; i < N; i += blockDim.x * gridDim.x)
```

Listing 1: Grid-Stride Loop implementatie

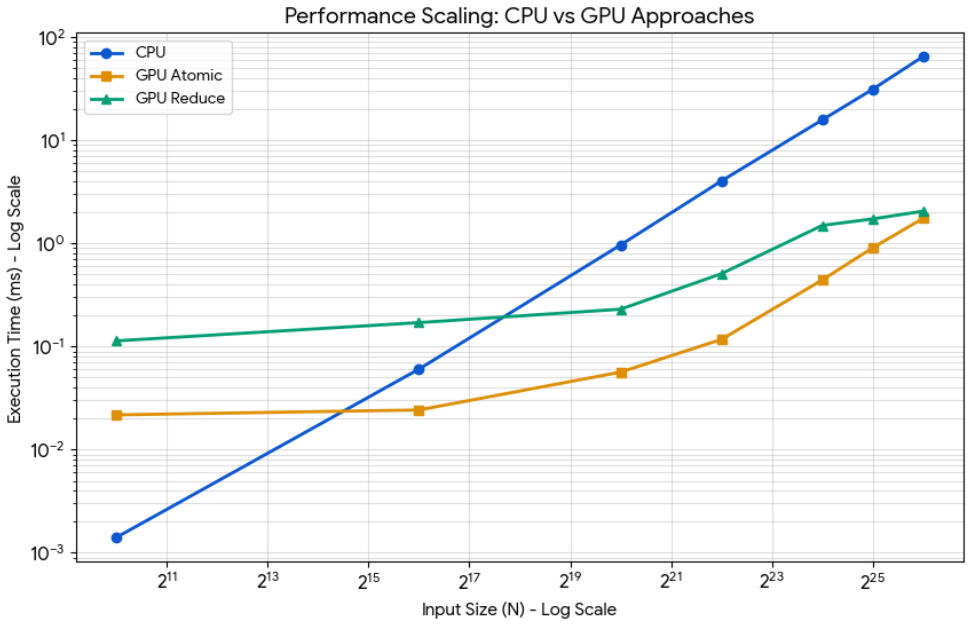
In plaats van dat een thread stopt na één element, springt hij met stappen ter grootte van de grid door de array heen. Dit garandeert dat alle elementen worden verwerkt, ongeacht de grootte van N.

3 Resultaten en Analyse

De code is gebenchmarkt over een bereik van $N = 1.024$ tot $N = 67.108.864$ (2^{26}).

3.1 Scaling Analysis (Time vs. Input Size)

De onderstaande grafiek toont de executietijd (logaritmisch) als functie van de inputgrootte.



Figuur 1: Scaling Analysis: Executietijd vs Inputgrootte (Log-Log schaal).

Observaties:

- **CPU (Blauw):** Toont een perfect lineair verloop ($\mathcal{O}(N)$). De executietijd loopt op tot ca. 60ms bij $N = 67M$.
- **GPU (Oranje/Groen):**
  - **Latentie-drempel:** Bij kleine inputs ($N < 65.536$) blijft de GPU-tijd vlak en zelfs hoger dan CPU. Hier domineert de overhead van het opstarten van de kernel over het eigenlijke rekenwerk.
  - **Throughput-fase:** Bij grote inputs ($N > 1M$) stijgt de tijd lineair, maar veel langzamer dan de CPU.

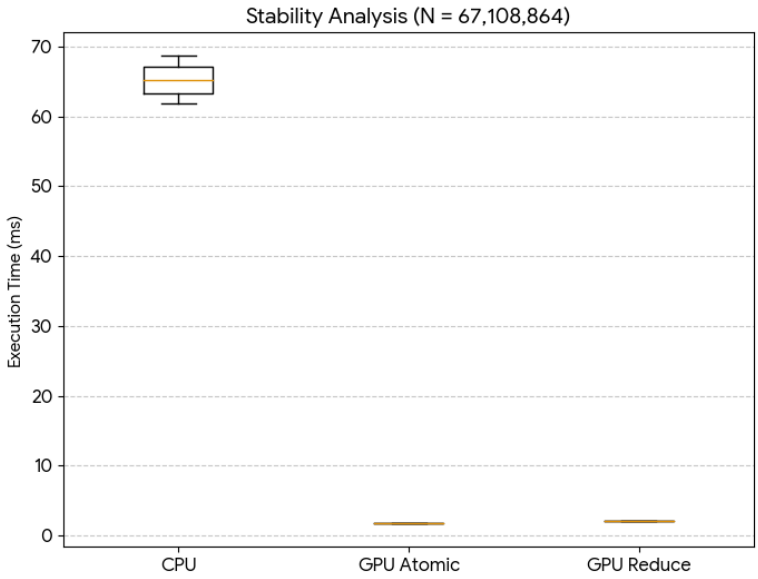
3.2 Atomic vs. Reduction Analyse

Opvallend in de grafiek hierboven is dat de **Atomic** methode (Oranje) consequent iets sneller presteert dan de **Reduction** methode (Groen). Dit lijkt in te gaan tegen de theorie dat reductie efficiënter is.

- **Oorzaak:** In de implementatie worden `cudaMalloc` en `cudaFree` aangeroepen *binnen* de tijdsmeting voor de tussenliggende buffers. Geheugenallocatie is een dure operatie.
- **Hardware:** Moderne GPU's handelen atomics af in de L2-cache, waardoor *contention* veel minder vertragend werkt dan op oudere hardware.

3.3 Stabiliteitsanalyse

De onderstaande Box Plot toont de spreiding van de executietijd bij maximale belasting ($N \approx 67M$) over meerdere runs.



Figuur 2: Stability Analysis: Box plot van executietijden bij $N \approx 67M$.

De CPU toont een grotere variantie door invloed van het OS (Jitter). De GPU toont zeer compacte boxen, wat bewijst dat de hardware zeer deterministisch presteert zodra de kernel gelanceerd is.

4 Antwoorden op Guiding Questions

4.1 Is the result correct?

Ja. Zoals besproken in sectie 3 en gevalideerd door de output in de benchmark, komen de resultaten van zowel de Atomic als de Reduction kernel exact overeen met de CPU-baseline.

4.2 How does time increase as a function of input size?

Zoals getoond in Figuur 1 (Sectie 3.1), schaalt de tijd op de CPU lineair ($\mathcal{O}(N)$). De GPU vertoont eerst een vlakke latentie-fase, waarna de tijd lineair toeneemt met een veel kleinere helling dan de CPU.

4.3 What happens if the input is larger than 2048?

Omdat `__syncthreads()` enkel werkt binnen een thread block (beperkt tot max. 1024 threads), is globale synchronisatie nodig voor grotere inputs. Dit is opgelost door de kernel recursief aan te roepen vanuit de host (CPU). Elke kernel-launch fungeert als een impliciete globale synchronisatiebarrière.

4.4 Consider calling the kernel with much less threads than data elements.

Dit wordt opgevangen door de **Grid-Stride Loop** die is uitgelegd in Sectie 2.3. Hierdoor verwerkt elke thread meerdere data-elementen totdat de volledige array doorlopen is.

4.5 How would you tackle a 2D array?

Er zijn twee methoden om een 2D array ($N \times M$) te verwerken:

- **Flattening:** De array in het geheugen beschouwen als 1D array ter grootte van $N \cdot M$ en de huidige kernels ongewijzigd gebruiken.
- **Row Mapping:** De Y-dimensie van de grid mappen op de rijen van de matrix (`blockIdx.y`). Elk block reduceert één rij, waarna een tweede reductiestap de resultaten van alle rijen combineert.

5 Conclusie

Dit experiment toont aan dat GPU's een significante versnelling bieden voor max-detectie op grote datasets in vergelijking met CPU-implementaties. Hoewel parallelle reductie-algoritmen in theorie een betere schaalbaarheid ($O(\log N)$) bieden, wijzen de resultaten uit dat het gebruik van `atomicMax` op moderne GPU-architecturen een uiterst efficiënt alternatief is. Dit geldt met name wanneer de implementatie-complexiteit en de overhead van geheugenallocatie en synchronisatie bij multi-pass reducties in overweging worden genomen. De hardwarematige optimalisaties van atomics op recente videokaarten maken deze aanpak in veel praktische scenario's superieur.

A Bijlage: Broncode

```
#include <cstdio>
#include <cstdlib>
#include <cstdint>
#include <climits>
#include <vector>
#include <random>
#include <chrono>
#include <iostream>
#include <cuda_runtime.h>

#define CUDA_CHECK(x) do { cudaError_t e=(x); if(e!=cudaSuccess){ \
  fprintf(stderr,"CUDA ERROR %s:%d: %s\n",__FILE__,__LINE__,cudaGetErrorString(e)); \
  std::exit(EXIT_FAILURE);} } while(0)


__host__ __device__ inline int i_max(int a, int b) { return (a > b) ? a : b; }
__host__        inline int i_min(int a, int b) { return (a < b) ? a : b; }

int cpuMax(const int* data, int N) {
    int m = INT_MIN;
    for (int i = 0; i < N; ++i) if (data[i] > m) m = data[i];
    return m;
}

__global__ void kernelAtomicMax(const int* d_in, int* d_gMax, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    for (int i = idx; i < N; i += blockDim.x * gridDim.x) {
        atomicMax(d_gMax, d_in[i]);
    }
}

__global__ void kernelReduceBlockMax(const int* d_in, int* d_blockMax, int N) {
    extern __shared__ int sdata[];
    int tid  = threadIdx.x;
    int idx  = blockIdx.x * blockDim.x + tid;

    int tmax = INT_MIN;
    for (int i = idx; i < N; i += blockDim.x * gridDim.x)
        tmax = i_max(tmax, d_in[i]);

    sdata[tid] = tmax;
    __syncthreads();

    for (int stride = blockDim.x >> 1; stride > 0; stride >>= 1) {
        if (tid < stride) sdata[tid] = i_max(sdata[tid], sdata[tid + stride]);
        __syncthreads();
    }
    if (tid == 0) d_blockMax[blockIdx.x] = sdata[0];
}

int gpuReduceMax(const int* d_in, int N, int tpb) {
    int blocks = i_min(65535, (N + tpb - 1) / tpb);
    if (blocks < 1) blocks = 1;

    int* d_buf = nullptr; CUDA_CHECK(cudaMalloc(&d_buf, blocks * sizeof(int)));
    size_t shmem = tpb * sizeof(int);

    kernelReduceBlockMax<<<blocks, tpb, shmem>>>(d_in, d_buf, N);
    CUDA_CHECK(cudaGetLastError());

```

```
61      int n = blocks; int* d_curr = d_buf;
62      while (n > 1) {
63          int nb = i_min(65535, (n + tpb - 1) / tpb);
64          if (nb < 1) nb = 1;
65          int* d_next = nullptr; CUDA_CHECK(cudaMalloc(&d_next, nb * sizeof(int)));
66          kernelReduceBlockMax<<<nb, tpb, shmem>>>(d_curr, d_next, n);
67          CUDA_CHECK(cudaGetLastError());
68          CUDA_CHECK(cudaFree(d_curr));
69          d_curr = d_next; n = nb;
70      }
71      int h; CUDA_CHECK(cudaMemcpy(&h, d_curr, sizeof(int), cudaMemcpyDeviceToHost));
72      CUDA_CHECK(cudaFree(d_curr));
73      return h;
74  }
75
76
77
78  int main() {
79      std::vector<int> test_sizes = {
80          1024,           // 1K
81          65536,          // 64K
82          1048576,        // 1M
83          4194304,        // 4M
84          16777216,       // 16M
85          33554432,       // 32M
86          67108864        // 64M
87      };
88
89      int tpb = 256;
90
91      // Warm-up run (om driver overhead weg te werken)
92      std::cout << "Performing GPU Warm-up...\n";
93      {
94          int N_warm = 1024;
95          int* d_dumb; cudaMalloc(&d_dumb, N_warm*sizeof(int));
96          gpuReduceMax(d_dumb, N_warm, tpb); // Fake run
97          cudaFree(d_dumb);
98      }
99      std::cout << "Warm-up done. Starting Benchmark...\n\n";
100
101         std::cout << "N,CPU_ms,Atomic_ms,Reduce_ms,Correct\n";
102
103     for (int N : test_sizes) {
104         // Data setup
105         std::vector<int> h_in(N);
106         std::mt19937 rng(42);
107         std::uniform_int_distribution<int> dist(-1000000000, 1000000000);
108
109         for (int i = 0; i < N; ++i) h_in[i] = dist(rng);
110         h_in[N/2] = 2000000000; // Forceer een max value ergens in het midden
111
112         // 1. CPU Timing
113         auto t0 = std::chrono::high_resolution_clock::now();
114         int cpu_res = cpuMax(h_in.data(), N);
115         auto t1 = std::chrono::high_resolution_clock::now();
116         double cpu_ms = std::chrono::duration<double, std::milli>(t1 - t0).count();
117
118         // GPU Setup
119         int* d_in = nullptr;
120         CUDA_CHECK(cudaMalloc(&d_in, N * sizeof(int)));
121         CUDA_CHECK(cudaMemcpy(d_in, h_in.data(), N * sizeof(int),
                cudaMemcpyHostToDevice));
122
```

```
        // 2. Atomic Timing
        int* d_gMax = nullptr;
        CUDA_CHECK(cudaMalloc(&d_gMax, sizeof(int)));
        int initMin = INT_MIN;
        CUDA_CHECK(cudaMemcpy(d_gMax, &initMin, sizeof(int), cudaMemcpyHostToDevice)
            );

        int blocks = i_min(65535, (N + tpb - 1) / tpb);

        cudaEvent_t aStart, aStop;
        cudaEventCreate(&aStart); cudaEventCreate(&aStop);

        cudaEventRecord(aStart);
        kernelAtomicMax<<<blocks, tpb>>>(d_in, d_gMax, N);
        cudaEventRecord(aStop);
        cudaEventSynchronize(aStop);

        float atomic_ms = 0.0f;
        cudaEventElapsedTime(&atomic_ms, aStart, aStop);

        int atomic_res;
        cudaMemcpy(&atomic_res, d_gMax, sizeof(int), cudaMemcpyDeviceToHost);

        // 3. Reduce Timing
        cudaEvent_t rStart, rStop;
        cudaEventCreate(&rStart); cudaEventCreate(&rStop);

        cudaEventRecord(rStart);
        int reduce_res = gpuReduceMax(d_in, N, tpb);
        cudaEventRecord(rStop);
        cudaEventSynchronize(rStop);

        float reduce_ms = 0.0f;
        cudaEventElapsedTime(&reduce_ms, rStart, rStop);

        // Check Correctness
        bool ok = (atomic_res == cpu_res) && (reduce_res == cpu_res);


        std::cout << N << ","
                  << cpu_ms << ","
                  << atomic_ms << ","
                  << reduce_ms << ","
                  << (ok ? "YES" : "NO") << "\n";

        cudaFree(d_gMax);
        cudaFree(d_in);
        cudaEventDestroy(aStart); cudaEventDestroy(aStop);
        cudaEventDestroy(rStart); cudaEventDestroy(rStop);
    }

    return 0;
}
```

Referenties

[1] NVIDIA Corporation. (2023). *CUDA C++ Programming Guide, Release 12.3*. Geraadpleegd via NVIDIA Developer Documentation.

[2] Google. (2025). *Gemini* . https://gemini.google.com